



CARDSAFE: ADVANCED FRAUD DETECTION AND ALERTS

Our Team

TEJA DURGA BHAVANI G

N190522

VIJAY LAKSHMI M

N190265

SIRISHA K

N190587

PRASUNA T

N190772

PRIYANKA K

N190832

TABLE OF CONTENTS

- ◆ Abstract
- ◆ Introduction
- ◆ Related works/ Existing works
- ◆ Proposed Method
- ◆ Technologies/Libraries used
- ◆ DataSet Used
- ◆ Visual Representations
- ◆ Conclusion
- ◆ Future Scope & Directions
- ◆ References

ABSTRACT

Cardsafe is an advanced system designed to detect and prevent credit card fraud using machine learning. It identifies unusual patterns, such as unexpected transaction locations or high-value purchases, to spot potential fraud. The system uses supervised learning to improve accuracy and minimize false alarms. Real-time alerts notify users and banks about suspicious activity through multiple communication channels. Key techniques include decision trees, logistic regression and random forests trained on real-world data. This system enhances security, reduces financial loss, and builds customer trust. Cardsafe offers a reliable, responsive solution for combating credit card fraud effectively.

INTRODUCTION

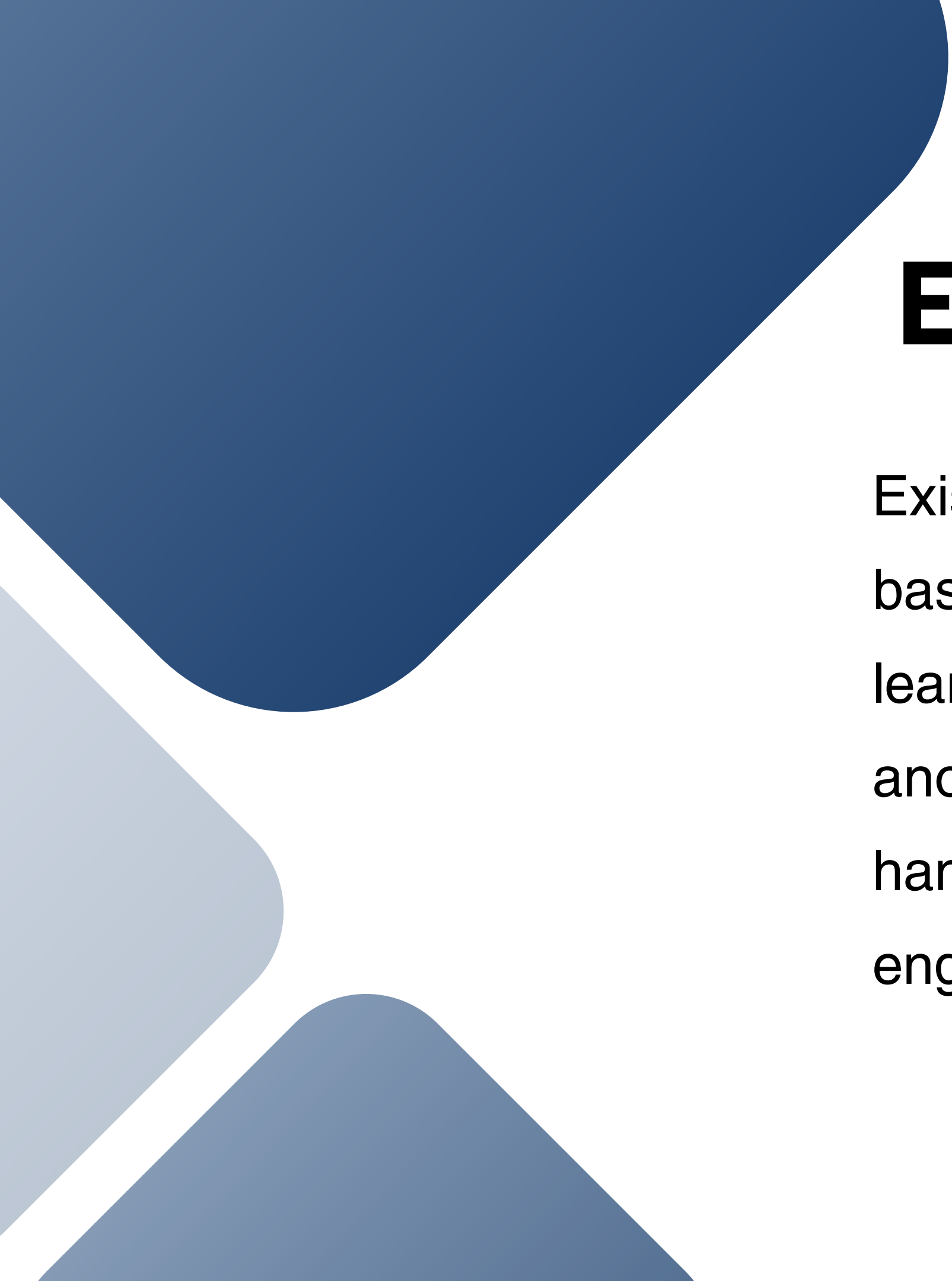
Cardsafe is an intelligent system that detects credit card fraud by analyzing real-time transaction patterns using advanced machine learning. It identifies unusual activities, such as transactions from unexpected locations or large purchases, by comparing them with a user's historical data. Upon detecting suspicious behavior, the system sends instant alerts to users and banks for swift action. This proactive approach helps prevent fraud, reduces financial losses, and builds user trust. Cardsafe also continuously adapts to new fraud techniques, ensuring ongoing effectiveness and providing a safer credit card experience.

MOTIVATION FOR THIS WORK

- ◆ **Rising Threat:** The increasing complexity and scale of online fraud demand robust detection methods.
- ◆ **Data-Driven Solutions:** Leveraging machine learning on transaction, user behavior, and device data offers more accurate and adaptive detection.
- ◆ **Real-Time Protection:** Real-time processing and user alerts ensure swift responses to prevent financial losses.
- ◆ **Enhanced Security:** This work aims to strengthen trust and security in digital transactions across e-commerce and financial sectors.

REAL WORLD APPLICATIONS

- ◆ **Banking: Real-time detection of unauthorized transactions.**
- ◆ **E-commerce: Preventing fraudulent purchases and account takeovers.**
- ◆ **Payment Gateways: Blocking suspicious transaction patterns.**
- ◆ **Subscription Services: Securing accounts against credential theft.**



EXISTING WORKS

Existing credit card fraud detection systems use rule-based models, logistic regression, and machine learning algorithms. They apply transaction profiling, anomaly detection, and sampling techniques to handle imbalanced data, along with feature engineering and visualization to enhance accuracy.



LIMITATIONS

Existing fraud detection systems face challenges such as limited labeled fraud data, high resource demands for real-time processing, and a tendency for high false-positive rates. Additionally, the need for frequent model updates to adapt to evolving fraud tactics adds complexity, limiting their scalability and long-term effectiveness.

PROPOSED APPROACH

The proposed system leverages libraries such as Random Forest for fraud detection, SMOTE to address class imbalance, Joblib for model serialization, and Matplotlib for data visualization. It features real-time user alerts to notify individuals immediately upon detecting suspicious activity, allowing for quick corrective actions. By combining these tools, the system enhances detection accuracy, minimizes fraud impact, and improves user confidence in secure digital transactions.

SOFTWARE REQUIREMENTS

Software Requirements:

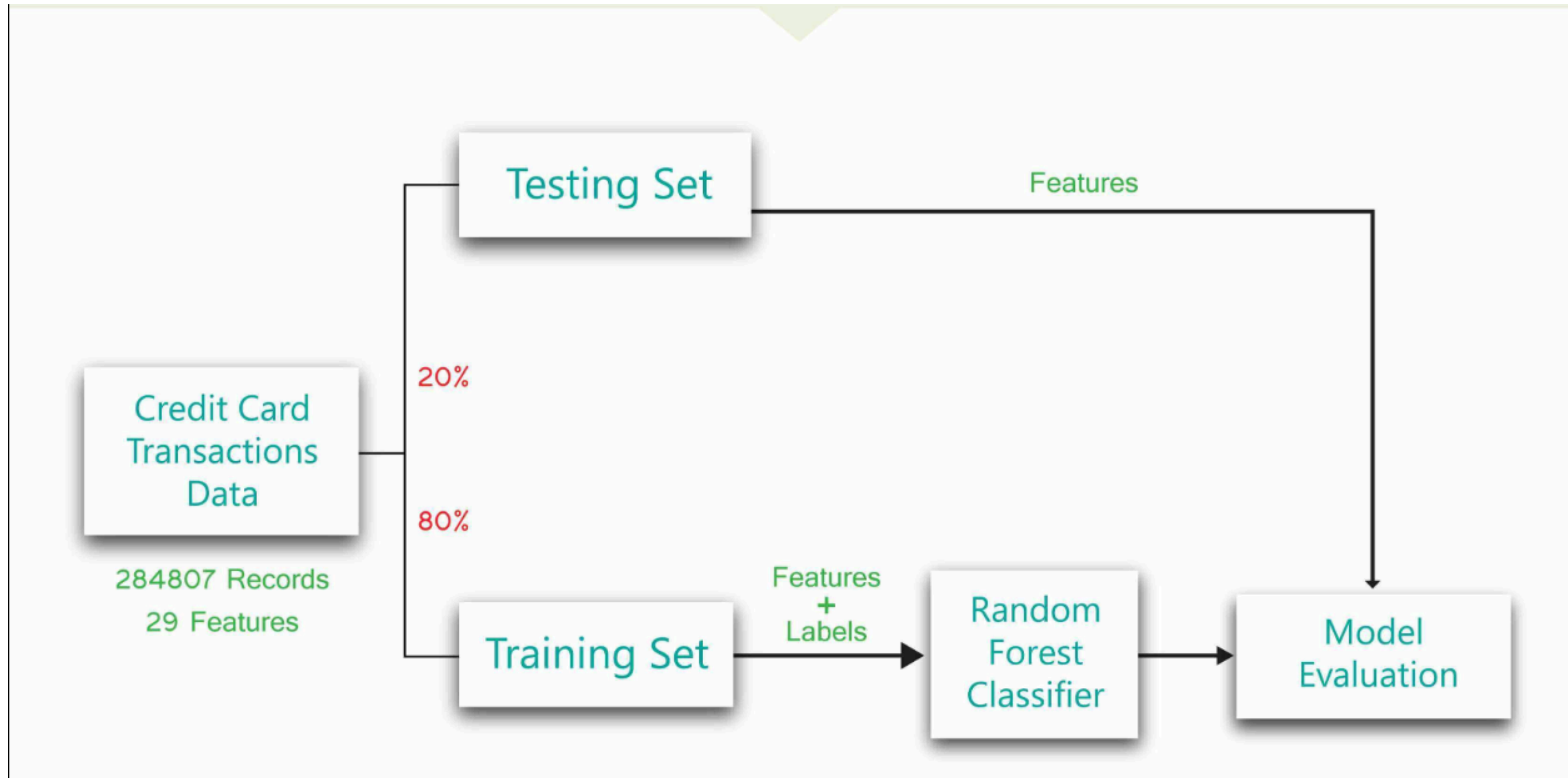
- Python 3.x: Required for developing and running the fraud detection system.
- pandas: For data manipulation and handling large datasets efficiently.
- matplotlib: For visualizing trends and patterns in transaction data.
- scikit-learn: For implementing machine learning algorithms and techniques like SMOTE.
- joblib: For serializing the trained models for deployment and scaling.

HARDWARE REQUIREMENTS

Hardware Requirements:

- Processor: A multi-core CPU (Intel i5 or equivalent) for fast data processing and real-time detection.
- RAM: Minimum 8 GB RAM for handling large datasets and running machine learning models.
- Storage: At least 50 GB of free disk space for storing datasets, models, and logs.
- Graphics Processing Unit (GPU): A GPU (NVIDIA or equivalent) recommended for deep learning model training.
- Network: Stable internet connection for data transfer, real-time alerts, and model updates.
-

FLOW CHART



WORK FLOW OF FLOW CHART

- Data Split: The dataset is divided into training (80%) and testing (20%) sets.
- Model Training: A Random Forest Classifier is trained on the training set.
- Model Evaluation: The trained model is evaluated on the testing set using metrics like accuracy, precision, recall, and F1-score.
- Fraud Detection: The model is deployed to detect fraudulent transactions in real-time, triggering alerts or blocking suspicious activity.

TECHNOLOGIES/LIBRARIES USED

Joblib:

- Purpose: Efficient serialization of Python objects, particularly machine learning models.
- Features: Provides fast saving and loading of large models, ensuring scalable deployment and reducing re-training time. It is optimized for handling large datasets and complex models in machine learning projects.

TECHNOLOGIES/LIBRARIES USED

SMOTE:

- Purpose: Addressing class imbalance by generating synthetic samples for the minority class.
- Features: Creates new instances by interpolating between existing minority class samples, balancing the dataset and improving the model's ability to detect rare events like fraud.

Pandas:

- Purpose: Data manipulation and analysis.
- Features: Handling structured data in the form of dataframes, making it easier to manage and analyze large datasets.

TECHNOLOGIES/LIBRARIES USED

Matplotlib and Seaborn:

- Purpose: Data visualization.
- Features: Creating plots and charts to visualize sentiment trends across different product categories.

Scikit-learn:

- Purpose: Additional text preprocessing and feature extraction.
- Features: Tools like TF-IDF Vectorizer for text representation.

DATA COLLECTION

Dataset Description

Data collection involves transaction details, user behavior, device information, and location to identify fraud patterns. It includes structured data (e.g., amounts, timestamps) and unstructured data (e.g., behavior patterns), used for training machine learning models.

DATA PRE PROCESSING

Input Data:

The dataset includes features such as transaction amount, time, location, and anonymized details about the transaction. A labeled dataset often contains a "fraud" flag (1 for fraud, 0 for legitimate transactions).

Data Cleaning:

Handle missing values, outliers, and anomalies in the dataset.

Feature Scaling: Normalize or standardize numerical features.

Balancing the Dataset: Use techniques like oversampling (SMOTE) or undersampling to address class imbalance (fraud cases are rare).

DATA CLASSIFICATION

Data Classification

Fraudulent Data:

1 (fraudulent) if the probability exceeds a threshold.

Classified as fraudulent based on anomalies detected through Random Forest, Decision Tree, and Logistic Regression models, showing patterns of unusual behavior.

DATA CLASSIFICATION

Data Classification

Non-Fraudulent Data:

0 (legitimate) if it falls below the threshold.

Classified as non-fraudulent, representing normal transaction patterns that align with legitimate user activity.

OUTPUT

Data Set Analyzation

```
In [54]: pred[0]
```

```
Out[54]: 0
```

```
In [55]: if pred[0] == 0:  
         print("Normal Transcation")  
     else:  
         print("Fraud Transcation")
```

```
Normal Transcation
```

```
In [ ]:
```

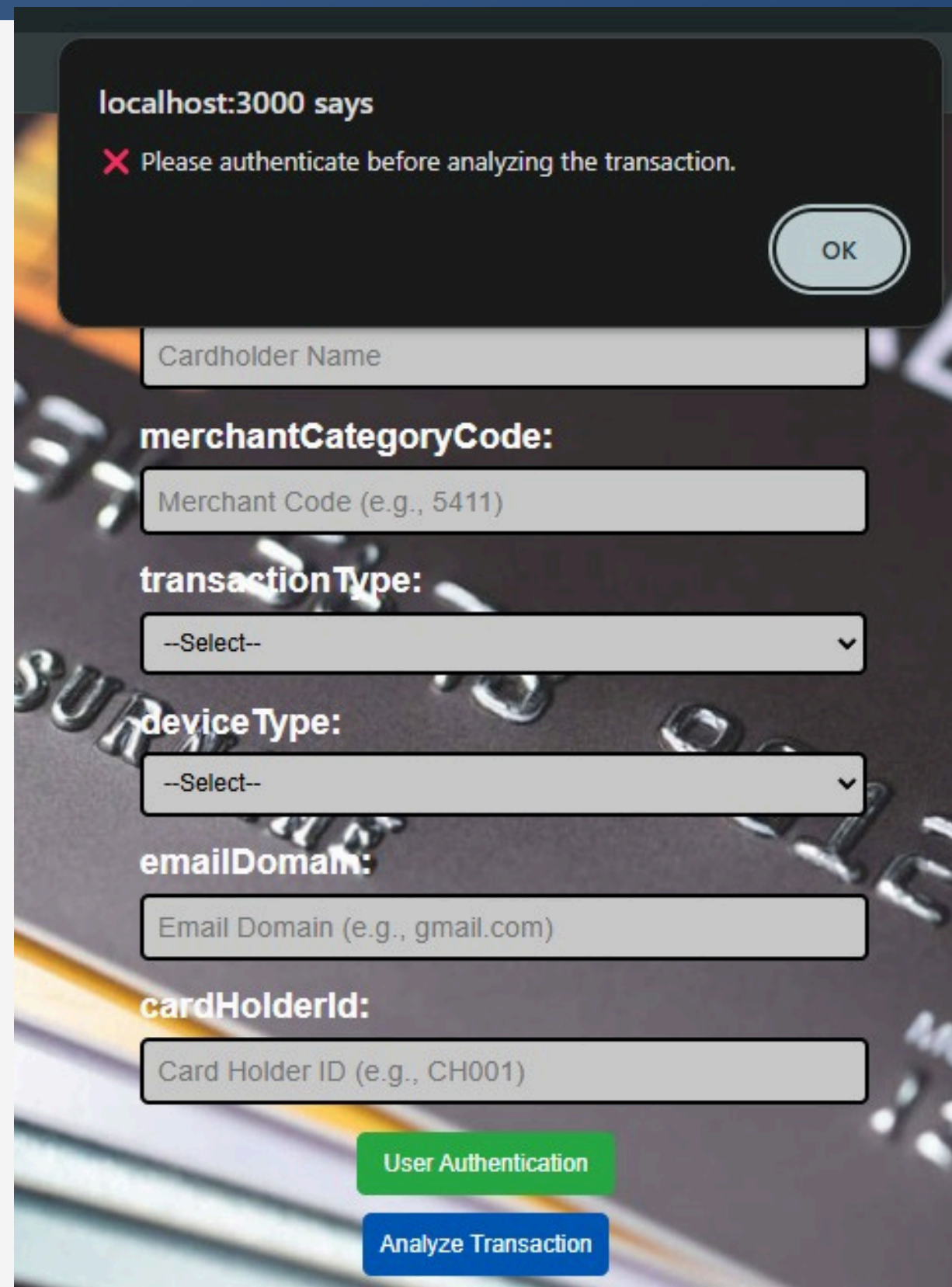
```
+ ✂ 📄 📁 ⬆ ⬇ ▶ Run ■ ↺ ⏩ Code ▼ 🖨
```

```
for name, clf in classifier.items():  
    print(f"\n===== {name} =====")  
    clf.fit(X_train, y_train)  
    y_pred = clf.predict(X_test)  
    print(f"\n Accuaracy: {accuracy_score(y_test, y_pred)}")  
    print(f"\n Precision: {precision_score(y_test, y_pred)}")  
    print(f"\n Recall: {recall_score(y_test, y_pred)}")  
    print(f"\n F1 Score: {f1_score(y_test, y_pred)}")  
  
=====Logistic Regression=====  
  
Accuaracy: 0.9473684210526315  
  
Precision: 0.9893617021276596  
  
Recall: 0.9117647058823529  
  
F1 Score: 0.9489795918367346  
  
=====Decision Tree Classifier=====  
  
Accuaracy: 0.9421052631578948  
  
Precision: 0.9504950495049505  
  
Recall: 0.9411764705882353  
  
F1 Score: 0.9458128078817734  
  
In [39]: # OVERSAMPLING  
  
In [40]: X = data.drop('Class', axis = 1)  
         y= data['Class']  
  
In [41]: X.shape
```

EXTENSION

As an extension, the project incorporates real-time fraud detection, enabling the system to monitor transactions continuously. The trained model is deployed to analyze transactions instantly, identifying anomalies as they occur. If a transaction is flagged as fraudulent, the system triggers immediate alerts to notify users or administrators. This proactive approach enhances security by allowing swift intervention, minimizing potential losses.

OUTPUT



localhost:3000 says

✖ Please authenticate before analyzing the transaction.

OK

Cardholder Name

merchantCategoryCode:

Merchant Code (e.g., 5411)

transactionType:

--Select--

deviceType:

--Select--

emailDomain:

Email Domain (e.g., gmail.com)

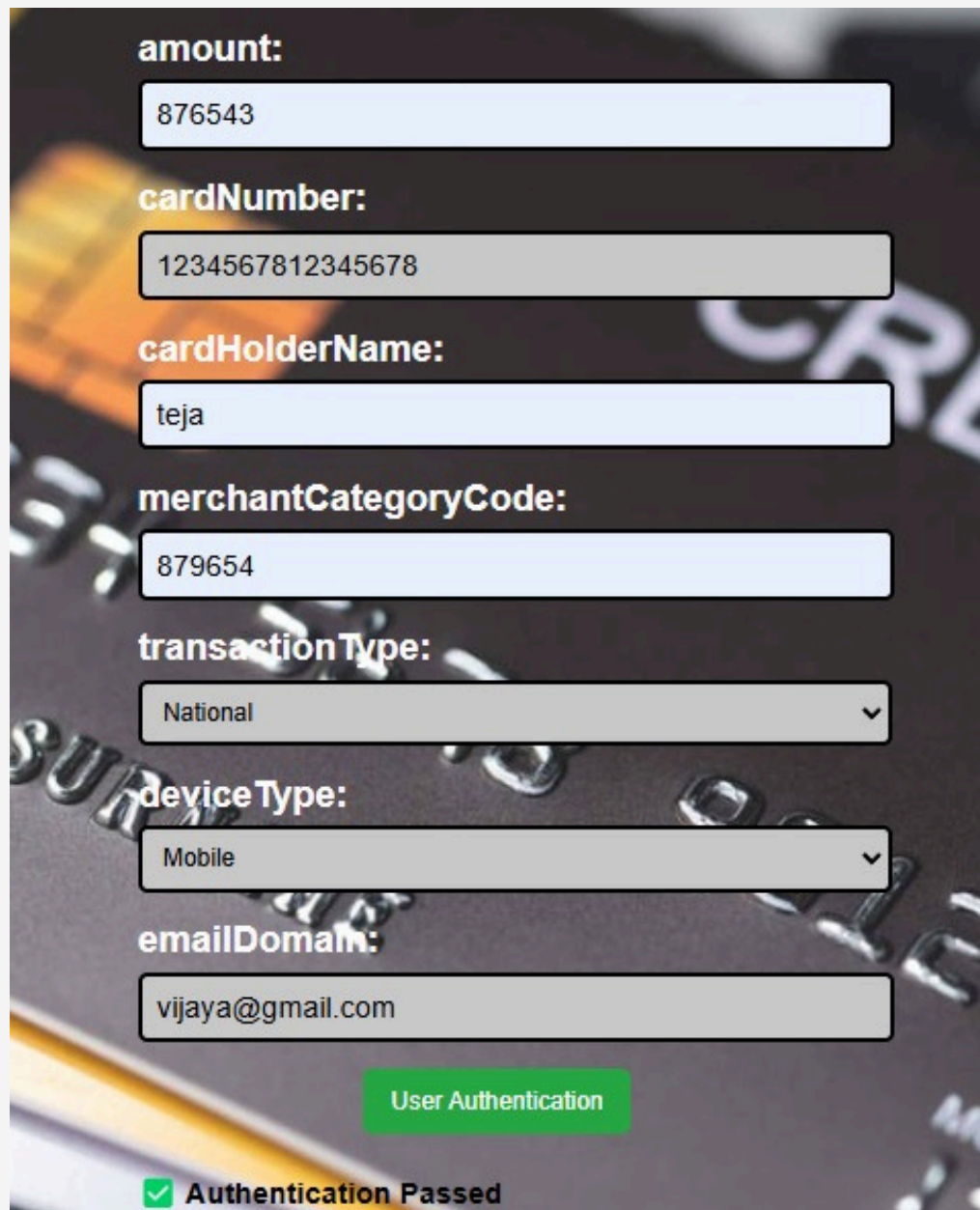
cardHolderId:

Card Holder ID (e.g., CH001)

User Authentication

Analyze Transaction

OUTPUT

A user authentication form overlaid on a background image of a credit card. The form contains several input fields for transaction details and a green button for authentication. At the bottom, a green checkmark and the text 'Authentication Passed' indicate a successful login.

amount:
876543

cardNumber:
1234567812345678

cardHolderName:
teja

merchantCategoryCode:
879654

transactionType:
National ▼

deviceType:
Mobile ▼

emailDomain:
vijaya@gmail.com

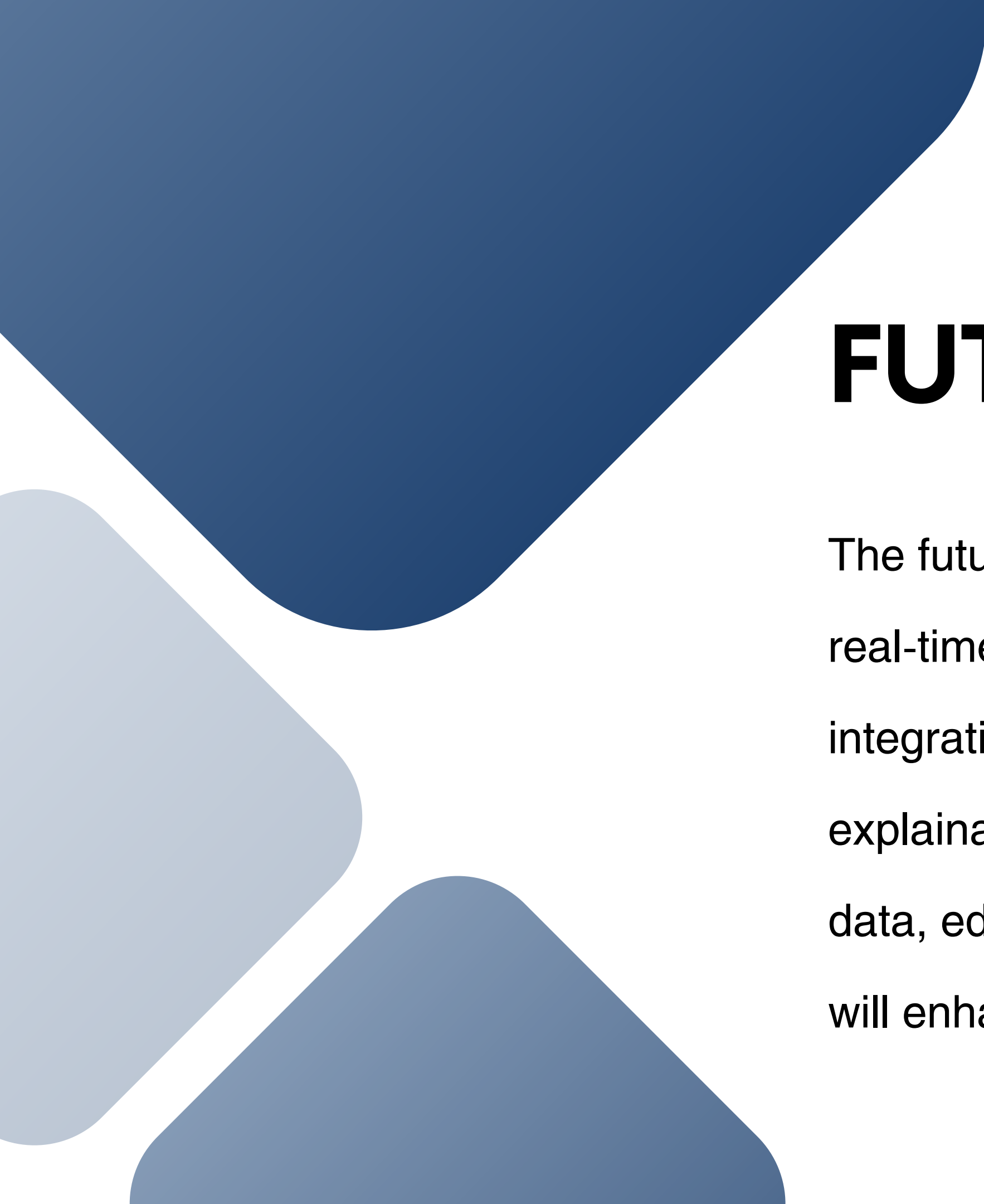
User Authentication

✓ Authentication Passed



CONCLUSION

The Cardsafe system effectively addresses the growing challenge of credit card fraud by leveraging advanced machine learning techniques and real-time transaction monitoring. By accurately detecting suspicious activities and sending instant alerts, it minimizes financial losses and protects users from unauthorized transactions. The system's adaptive learning improves accuracy over time, ensuring it stays ahead of evolving fraud tactics. Cardsafe not only enhances security for financial institutions and cardholders but also builds trust in digital transactions, offering a reliable and proactive solution for combating credit card fraud.



FUTURE SCOPE

The future scope of credit card fraud detection includes real-time detection, adaptive models, blockchain integration, federated learning for privacy, and explainable AI. Combining behavioral biometrics with data, edge computing, and cross-industry collaboration will enhance accuracy and prevention strategies.



THANK YOU

REFERENCES

- Kaggle Credit Card Fraud Detection Dataset:
<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- IEEE Paper reference:
<https://ieeexplore.ieee.org/document/9121114>
- IEEE Paper reference:
<https://ieeexplore.ieee.org/document/10699070>
- IEEE Paper reference:
<https://ieeexplore.ieee.org/document/10735025>